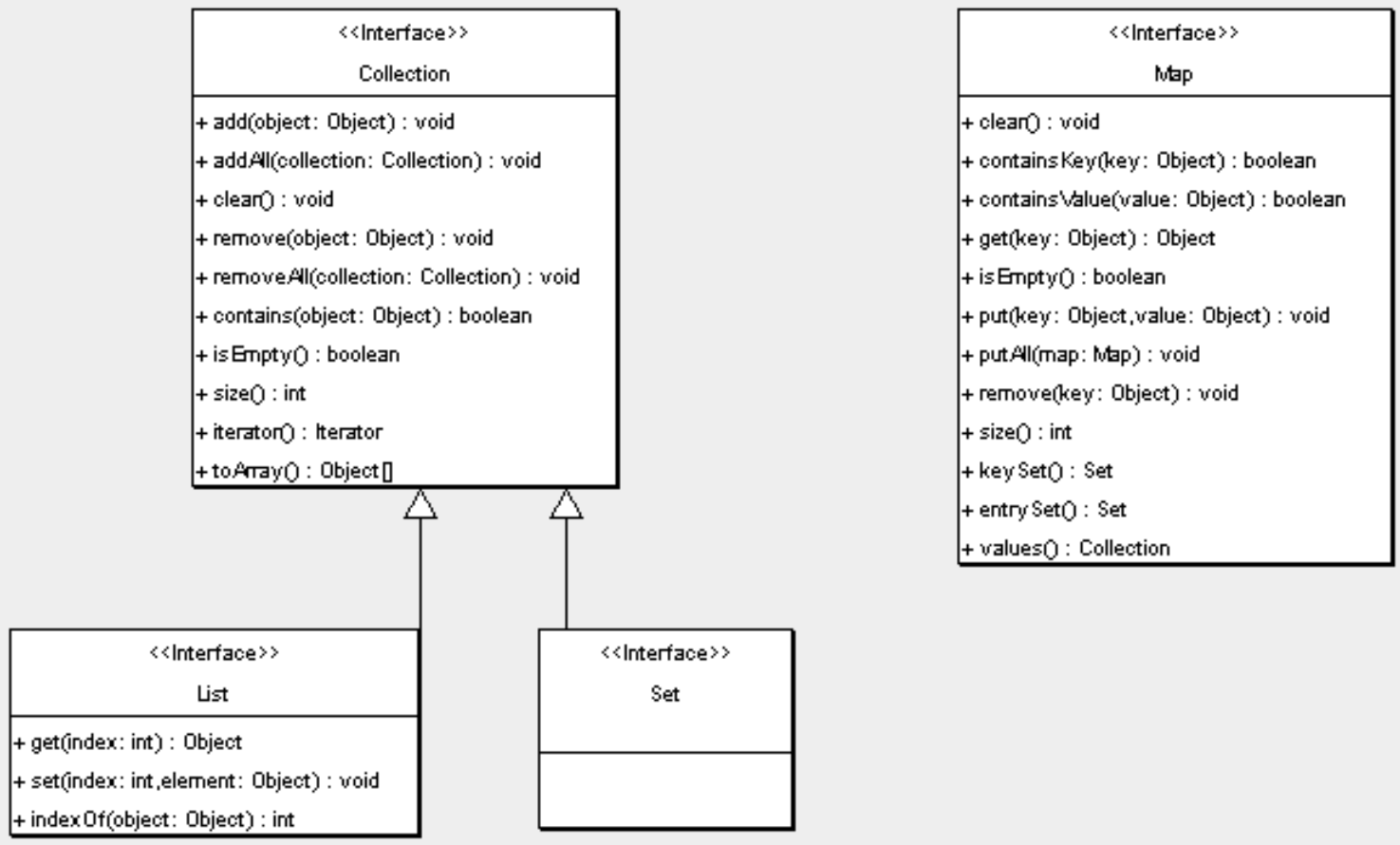
Col·leccions: Més enllà de les taules

PROP 2021-2022 Q2

Mario Martin – Ignasi Gómez Sebastià – Bernat Orellana

Collection<T>: Mètodes de la interfície



Exercicis Llistes

- Crear una classe que implementi una cua a partir d'una ArrayList
  - `Crear cua buida`
  - `Afegir element`
  - `Obtenir un element de la cua`
    - `Returneu null si la cua es buida`
- Crear una classe que implementi una pila a partir d'una LinkedList
  - `Crear pila buida`
  - `Afegir element`
  - `Obtenir un element de la pila`
    - `Returneu null si la cua es buida`

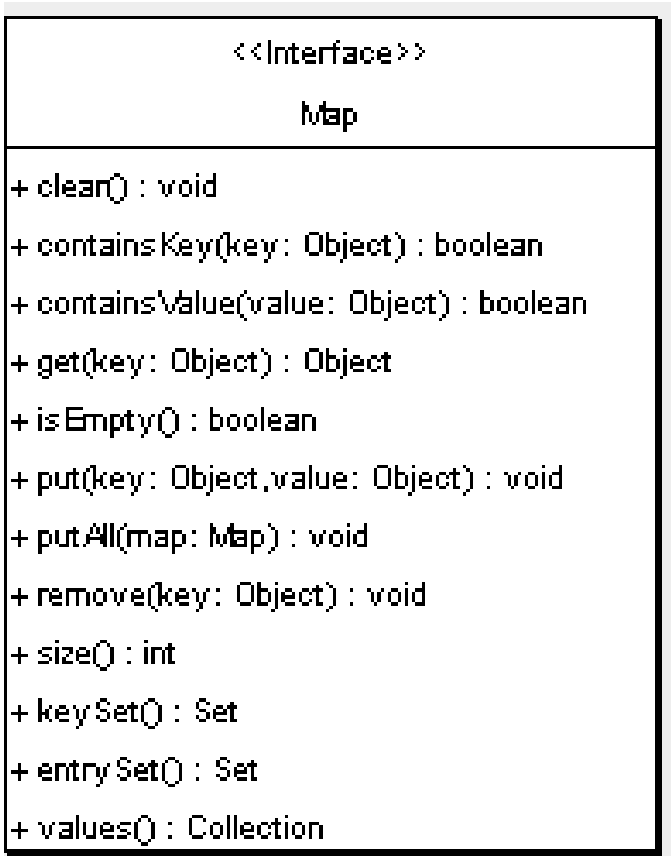
Maps

Una agrupació d'objectes que no és una col·lecció

Maps

- Implementació de memòria associativa
- Parelles <clau,valor>
- S'obté el valor associat a una clau no pel índex sinó per un càlcul (de hash) sobre la clau.
- Per exemple. Volem guardar freqüències de paraules
  - <String, Integer>
  - Accedim a nombre de vegades que apareix la paraula per la mateixa paraula (no per un índex com en una taula)

Map<K,V>: Mètodes de la interfície



Mètodes de la interfície

clear()	Elimina totes les associacions
containsKey(k)	Si conté una assoc. per a k
containsValue(v)	Si conté una assoc. per a v
get(k)	Valor associat amb k (o null)
isEmpty()	Si esta buit
put(k,v)	Associar v amb k
remove(k)	Eliminar associacions per a k
size()	Nombre de parelles
keySet()	Conjunt de claus
values()	Col·lecció de valors

Implementacions de Map<K,V>

- Diverses implementacions de la interfície **`Map<T>`:**
  - **`HashMap`:** Implementació del Map amb taules de hast. *[Més usat]*
  - **`TreeMap`:** Implementació del Map amb arbres balancejats. Manté un ordre.
- Exemple:

```
Map<String,Integer> noms = new HashMap<String,Integer>;
Map map = new HashMap();
```

Us del Map per a classes

- Per poder fer servir el Map, la classe que es fa servir com a clau ha de tenir implementats de els mètodes:

```
equals() ( per defecte compara punters )
hashCode() ( hash punters )
```

- En les classes habituals estan implementades, en les pròpies s'han de implementar

- Condició de coherència:
  - Donats dos objectes A, B,

    A.equals(B)==true ➔ A.hashCode()==B.hashCode()

    "Si dos objectes són iguals, han de tenir el mateix hash."

    NOTA: Al revés no cal que sigui cert.

Us del Map per a classes

```
public class Person {
    public String name;
    boolean equals(Object o) {
        return (o instanceof Person && ((Person)o).name.equals(name));
    }
    public int hashCode() {
        return name.hashCode();
    }
}
```

Exercicis Mapa

- Fer un programa que llegeix un text per el canal d'entrada estàndard i mostra per pantalla la freqüència de cada paraula diferent del text
- `Fer servir un mapa clau valor per guardar paraules i freqüències`

Iterations

Recorrent les col·leccions

Iteradors. Recorrent les col·leccions

- Les col·leccions i els Maps són extensions de la classe iterables.
- A la classe hi ha mètodes definits per recórrer els elements que componen l'objecte de la classe

Interfície Iteration

- Mètodes implementats

```
boolean hasNext( );
Object next( );
void remove( );
```

- Mètodes implementats addicionals pels ListIteration

```
boolean hasPrevious();
Object previous();
int nextIndex( );
int previousIndex( );
```

Us dels iteradors

- Exemple de mètode que imprimeix tots els elements d'una col·lecció:

```
static void printAll (Collection coll) {
 Iterator iter = coll.iterator( );
 while (iter.hasNext( )) {
    System.out.println(iter.next( ) );
 }
}
```

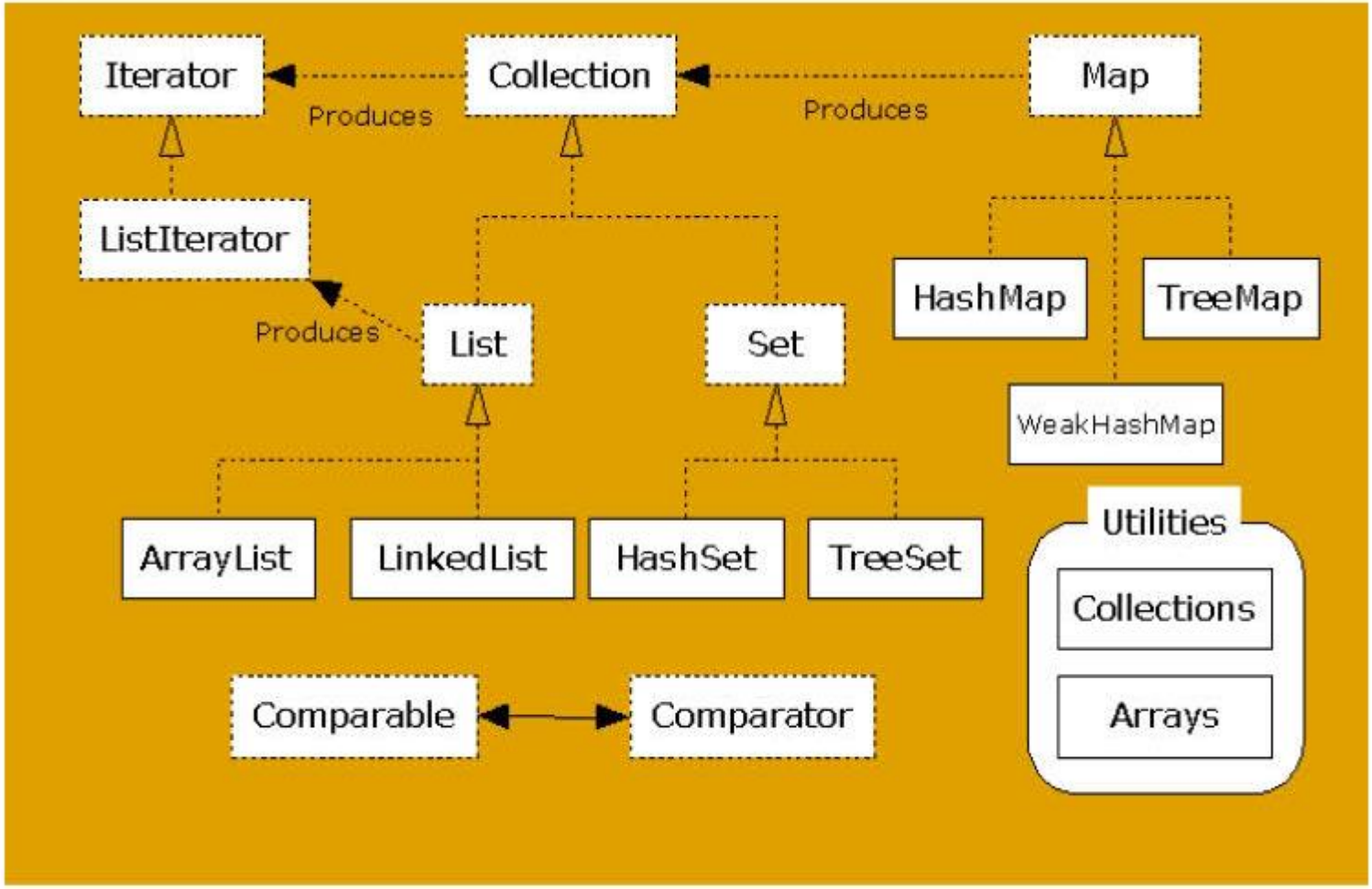
- Observeu que el codi és polimòrfic, és a dir, que funcionarà per qualsevol col·lecció.
- Hi ha constructores de iteradors específics per llistes:

```
ListIterator listIterator( );
ListIterator listIterator(int index);
```

la darrera comença en la posició *index* fins al final de la llista

Resum

Resum visual



Altres agrupacions

- Cues
- Piles
- ...

public interface Queue<E> extends Collection<E>

```
public interface Queue<E> extends
  Collection<E> {
     E element();
     E peek();
     boolean offer(E e);
     E remove();
     E poll();
}
```

Collections – Other Functions

- La classe `java.util.Collections` té molts mètodes útils per treballar amb col·leccions

Algorismes disponibles per llistes

- Són mètodes estàtics que implementen algorismes usuals en llistes:
  - sort: Ordena fent servir mergesort
  - shuffle: Barreja aleatòriament els elements de la llista
  - reverse: Inverteix l'ordre
  - rotate: Rota la llista el nombre de vegades que vulguem
  - fill: Reescriu tots els elements de la llista amb l'element que volem.
  - copy: Copia una llista sobre una altra
  - binarySearch: Cerca fent servir cerca binaria
  - indexOfSubList: Retorna el índex de la primera subllista trobada
  - lastIndexOfSubList: Retorna el índex de la darrera subllista trobada
  - max: Retorna el màxim de la llista
  - min: Retorna el mínim de la llista

Com definir l'ordre

- Alguns dels algorismes requereixen com per exemple la ordenació, buscar un màxim o mínim, etc. requereixen poder comparar els diferents elements de la col·lecció: cal definir un mecanisme de **comparació**.

- A tal efecte, ens caldrà programar un mètode que, donats 2 elements, retorni un enter:
  - Negatiu: quan el primer sigui menor que el segon
  - Zero: quan siguin iguals
  - Positiu

Com definir l'ordre: dues opcions

- **Aquest mètode es pot definir de dues maneres:**
  - **Ordre "natural" dels objectes:** Donada una classe de sobre la que volem ordenar/comparar (p.ex *Persona*), fem que implementi *Comparable*. Per tant, dins de la mateixa classe és defineix el comportament per defecte que s'usarà per comparar els seus objectes (p.ex. com es comparen les persones per defecte).
  - **Ordre "personalitzat" amb Comparator.** Ens pot interessar definir criteris d'ordenació diferents segons les circumstàncies. En aquest cas usem una tercera classe que exerceix les funcions de comparació: aquesta classe implemente *Comparator*. Podem tenir tants Comparators com diferents tipus d'ordre volguem tenir.

Ordre natural (i)

- La classe que volem comparar implementa *Comparable*, que ens suposa implementar alhora el mètode *compareTo()*

```
public class Persona implements Comparable {

    private int id;
    private String NIF;
    private String nom;
    private String cognoms;

    public Persona(int id, String NIF, String nom, String cognoms) {
        setId(id);
        setNom(nom);
        setCognoms(cognoms);
        setNIF(NIF);
    }

    /**
     * Ordre natural: comparem les persones per identificador
     * @param o
     * @return
     */
    @Override
    public int compareTo(Object o) {
        Persona p = (Persona) o;
        return this.id -p.id;
    }
```

Ordre natural (ii)

```
// Inicialitzem una llista de persones
List<Persona> persones = new LinkedList<>();
persones.add(new Persona(2, "22222222H", "Paco", "Jones"));
persones.add(new Persona(4, "44444444H", "Cris", "Tina"));
persones.add( new Persona(1, "11111111H", "Maria", "González"));
persones.add( new Persona(3, "33333333H", "Joan", "Capella"));

Collections.sort(persones); // Comparació usant l'ordre natural ( definit per Comparable )
System.out.println("=== Ordre natural (per ID) ====================");
for(Persona p:persones) {
    System.out.println(">"+p);
}
```

```
=== Ordre natural (per ID) ====================
>Persona{id=1, NIF=11111111H, nom=Maria, cognoms=González}
>Persona{id=2, NIF=22222222H, nom=Paco, cognoms=Jones}
>Persona{id=3, NIF=33333333H, nom=Joan, cognoms=Capella}
>Persona{id=4, NIF=44444444H, nom=Cris, cognoms=Tina}
```

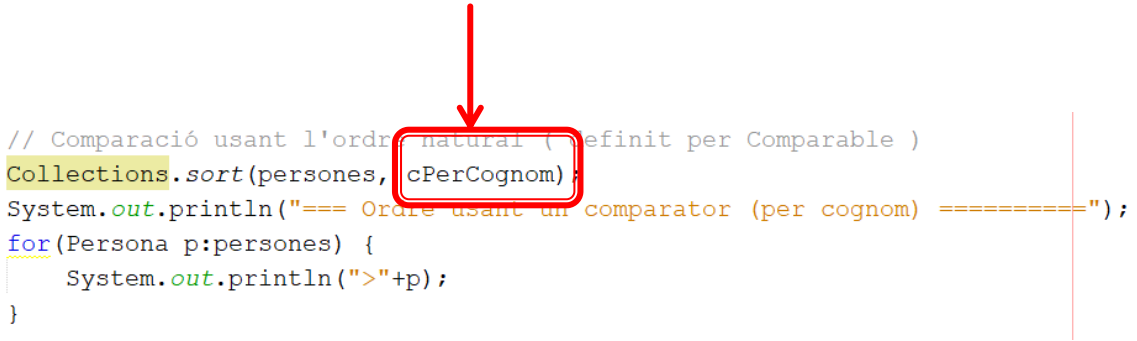
Ordre amb Comparator (i)

- Externament a la classe persona, definim una classe que implementi *Comparator*. En el mètode *compare* programem el criteri de comparació que ens interessi, en aquest cas la comparació per cognom.

```
Comparator cPerCognom = new Comparator() {
    @Override
    public int compare(Object t, Object t1) {
        if(
                t!=null &&
                t1!=null &&
                t instanceof Persona &&
                t1 instanceof Persona) {
            Persona p1 = (Persona)t;
            Persona p2 = (Persona)t1;
            return p1.getCognoms().compareTo(p2.getCognoms());
        }
        return 0;
    }
};
```

Ordre amb Comparator (ii)

- Per usar el comparator, li passem com a paràmetre addicional a l'algorisme que el requereixi.



```
// Comparació usant l'ordre natural ( definit per Comparable )
Collections.sort(persones, cPerCognom);
System.out.println("=== Ordre usant un comparator (per cognom) ==========");
for(Persona p:persones) {
    System.out.println(">"+p);
}
```

```
=== Ordre usant un comparator (per cognom) ==========
>Persona{id=3, NIF=33333333H, nom=Joan, cognoms=Capella}
>Persona{id=1, NIF=11111111H, nom=Maria, cognoms=González}
>Persona{id=2, NIF=22222222H, nom=Paco, cognoms=Jones}
>Persona{id=4, NIF=44444444H, nom=Cris, cognoms=Tina}
```

Exercici comparador

- Crear classe futbolista
  - Nom
  - Gols
  - Assistències
- Crear una col·lecció de futbolistes i:
- Ordenar per nom ascendent (ordre natural)
- Ordenar per nom descendent (ordre natural)
- Ordenar (useu un Comparator a mida):
  - Nombre de gols
  - En cas d'empat nombre d'assistències
  - En cas d'empat nom